UNIT-III

STACK

A **stack** is a non-primitive linear data structure. It is an ordered list in which addition of new data item and deletion of already existing data item is done from only one end, known as *Top of Stack (TOS)*. As all the deletion and insertion in a stack is done from top of the stack, the last added element will be the first to be removed from the stack. Due to this reason, the stack is also called **Last-In-First-Out (LIFO)** type of list.

SEQUENTIAL IMPLEMENTATION OF STACKS

Stack can be implemented in two ways :

(a) Static implementation
(b) Dynamic implementation
(c) Static implementation

Static implementation uses arrays to create stack. Static implementation is a very simple technique, but is not a flexible way of creation, as the size of stack has to be declared during program design, after that the size cannot be varied. Moreover, static implementation is not too efficient with respect to memory utilization. As the declaration of array (for implementing stack) is done before the start of the operation (at program design time), now if there are too few elements to be stored in the stack the statically allocated memory will be wasted, on the other hand if there are more number of elements to be stored in the stack then we can't be able to change the size of array to increase its capacity, so that it can accommodate new elements.

Dynamic implementation

As in static implementation, we have used arrays to store the elements that get added to the stack. However, implemented as an array it suffers from the basic limitation of array-that its size cannot be increased or decreased one it is declared. As a result, one ends up reserving either too much space or too less space for an array and in turn for a stack**. This problem can be overcome if we implement a stack using a linked list**. In case of a linked list we shall push and pop nodes from one end of a linked list. *Linked list representation* is commonly known as **Dynamic implementation** and uses pointers to implement the stack type of data structure. The stack as linked list is represented as a singly connected list. Each node in the linked list contains the data and a pointer that gives location of the next node in the list. The node in the list is a structure as shown below:

```
struct node
{
<data type> data;
node *link;
};
```

where <data type> indicates that the data can be of any type like int, float, char etc, and link, is a pointer to the next node in the list.

OPERATIONS ON STACK

The basic **operations** that can be performed on **stack** are as follows:

1. **PUSH :** The process of adding a new element to the top of the stack is called PUSH operation. Pushing an element in the stack involve adding of element, as the new element will be inserted at the top, so after every push operation, the top is incremented by one. In case the array is full and no new element can be accommodated, it is called STACK-FULL condition. This condition is called STACK OVERFLOW.
2. **POP :** The process of deleting an element from the top of the stack is called POP operation. After every pop operation, the stack is decremented by one. If there is no element on the stack and the pop is performed then this will result into STACK UNDERFLOW condition.

ALGORITHMS FOR PUSH & POP FOR STATIC IMPLEMENTATION USING ARRAYS

(i) Algorithm for inserting an item into the stack (PUSH)

Let STACK[MAXSIZE] is an array for implementing the stack, MAXSIZE represents the max. size of array STACK. NUM is the element to be pushed in stack & TOP is the index number of the element at the top of stack.

Step 1 : [Check for stack overflow ?]

If TOP = MAXSIZE – 1, then :

Write : 'Stack Overflow' and return.

[End of If Structure]

Step 2 : Read NUM to be pushed in stack.

Step 3 : Set TOP = TOP + 1 [Increases TOP by 1]

Step 4 : Set STACK[TOP] = NUM [Inserts new number NUM in new TOP Position]

Step 5 : Exit

The function of the Stack PUSH operation in C is as follows

```
void push()
{
if(top==MAXSIZE-1)
{
printf("\n\nStack is full(Stack overflow)"); return;
}
int num;
printf("\n\nEnter the element to be pushed in stack : ");
scanf("%d",&num);
top++;
```

```
stack[top]=num;
}
```

(i) Algorithm for deleting an item from the stack (POP)

Let STACK[MAXSIZE] is an array for implementing the stack where MAXSIZE represents the max. size of array STACK. NUM is the element to be popped from stack & TOP is the index number of the element at the top of stack.

Step 1 : [Check for stack underflow ?]

If TOP = -1 : then

Write : 'Stack underflow' and return.

[End of If Structure]

Step 2 : Set NUM = STACK[TOP]

[Assign Top element to NUM]

Step 3 : Write 'Element popped from stack is : ',NUM.

Step 4 : Set TOP = TOP - 1 [Decreases TOP by 1]

Step 5 : Exit

The function of the Stack POP operation in C is as follows :

```
void pop()
{
if(top== -1)
{
printf("\n\nStack is empty(Stack underflow)"); return;
}
int num; num=stack[top];
printf("\n\nElement popped from stack : %d",num); top--;
}
```

Implementation of Stack Operations Using Arrays

```
#include<stdio.h>
#include<stdlib.h>
```

```
#define MAXSIZE 10
void push();
void pop();
void display();
int stack[MAXSIZE];
int top=-1;
void main()
{
int choice;
 while(1)
 {
 printf("STATIC IMPLEMENTATION OF STACK");
 printf("\n********");
 printf("\n1. PUSH");
 printf("\n2. POP");
 printf("\n3. DISPLAY");
 printf("\n4. EXIT");
 printf("\n-----------------------------");
 printf("\n\nEnter your choice [1/2/3/4] : ");
 scanf("%d",&choice);
 switch(choice)
 {
 case 1 : push();
 break;
 case 2 : pop();
 break;
 case 3 : display();
 break;
 case 4 : exit(0);
 default : printf("\n\nInvalid choice");
 }
 }
}
// Function for the operation push
void push()
{
 int num,n,i;
 if(top==MAXSIZE-1)
 {
 printf("\n\nStack is full(Stack overflow)");
 return;
 }


 printf("Enter number of elements wants to insert");
 scanf("%d",&n);
 if(n<=MAXSIZE)
```

```
 {
 for(i=0;i<n;i++)
 {
 printf("\n\nEnter the element to be pushed in stack : ");
 scanf("%d",&num);
 top++;
 stack[top]=num;
}
}
else
{
    printf("number must be less than stack size");
}
}
// Function for the operation pop
void pop()
{
 int num;
 if(top==-1)
 {
 printf("\n\nStack is empty(Stack underflow)");
 return;
 }
 num=stack[top];
 printf("\n\nElement popped from stack : %d",num);
 top--;
}
// Function for traversing the stack
void display()
{
 if(top==-1)
 {
 printf("\n\nStack is empty(Stack underflow)");
 return;
 }
 printf("\n\nStack elements are : \n");
 for(int i=top;i>=0;i--)
 printf("Stack[%d] : %d\n",i,stack[i]);
}
```

Output:

```
STATIC IMPLEMENTATION OF STACK
1. PUSH
2. POP
3. DISPLAY
4. EXIT
```

```
-----------------------------
Enter your choice [1/2/3/4] : 1
Enter number of elements wants to insert: 3

Enter the element to be pushed in stack : 10

Enter the element to be pushed in stack : 20

Enter the element to be pushed in stack : 30

STATIC IMPLEMENTATION OF STACK
********
1. PUSH
2. POP
3. DISPLAY
4. EXIT
-----------------------------
Enter your choice [1/2/3/4] : 3

Stack elements are :
Stack[2] : 30
Stack[1] : 20
Stack[0] : 10

STATIC IMPLEMENTATION OF STACK
********
1. PUSH
2. POP
3. DISPLAY
4. EXIT
-----------------------------

Enter your choice [1/2/3/4] : 2

Element popped from stack : 30

STATIC IMPLEMENTATION OF STACK
********
1. PUSH
2. POP
3. DISPLAY
4. EXIT
-----------------------------
Enter your choice [1/2/3/4] : 3
```

```
Stack elements are :
Stack[1] : 20
Stack[0] : 10

STATIC IMPLEMENTATION OF STACK
********
1. PUSH
2. POP
3. DISPLAY
4. EXIT
-----------------------------
Enter your choice [1/2/3/4] : 4
```

Implementation of Stack using Linked List

```
#include <stdio.h>
#include <stdlib.h>
// Define a node structure
struct Node {
    int data;
    struct Node* next;
};
// Initialize top pointer
struct Node* top = NULL;
// Function to push an element onto the stack
void push(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Stack Overflow! No memory available.\n");
        return;
    }
    newNode->data = value;
    newNode->next = top;
    top = newNode;
    printf("%d pushed onto the stack.\n", value);
}
// Function to pop an element from the stack
void pop() {
    if (top == NULL) {
        printf("Stack Underflow! Stack is empty.\n");
        return;
    }
    struct Node* temp = top;
    printf("%d popped from the stack.\n", temp->data);
    top = top->next;
    free(temp);
}
```

```
// Function to peek at the top element of the stack
void peek() {
    if (top == NULL) {
        printf("Stack is empty.\n");
        return;
    }
    printf("Top element is %d\n", top->data);
}

// Function to display the stack elements
void display() {
    if (top == NULL) {
        printf("Stack is empty.\n");
        return;
    }
    struct Node* temp = top;
    printf("Stack elements: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice, value;
    while (1) {
        printf("\nStack Operations using Linked List\n");
        printf("1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;
            case 2:
                pop();
                break;
            case 3:
                peek();
                break;
            case 4:
                display();
                break;
```

```
            case 5:
                printf("Exiting program...\n");
                exit(0);
            default:
                printf("Invalid choice! Please try again.\n");
        }
    }
    return 0;
}
```

Output :

```
Stack Operations using Linked List
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 10
10 pushed onto the stack.

Enter your choice: 1
Enter value to push: 20
20 pushed onto the stack.

Enter your choice: 1
Enter value to push: 30
30 pushed onto the stack.

Enter your choice: 4
Stack elements: 30 -> 20 -> 10 -> NULL

Enter your choice: 3
Top element is 30

Enter your choice: 2
30 popped from the stack.

Enter your choice: 4
Stack elements: 20 -> 10 -> NULL

Enter your choice: 5
Exiting program...
```

Applications of Stacks:

1. Conversion from Infix to Postfix.
2. Conversion from Infix to Prefix
3. Postfix Evaluation
4. Validating the Parenthesis

Stacks are widely used in computer science and programming due to their LIFO (Last In, First Out) nature.

Expression Evaluation and Conversion:

- **Infix to Postfix/Prefix Conversion**: Stacks are used to convert expressions from infix notation (e.g., A + B) to postfix (e.g., A B +) or prefix, which simplifies evaluation.
- **Expression Evaluation**: Postfix (or prefix) expressions are evaluated using a stack, handling operators and operands efficiently.

Function Call Management:

Call Stack: When a function is called, its context (local variables, return address) is pushed onto the call stack. Once the function execution is complete, its context is popped off, allowing the program to return to the previous function.

Backtracking Algorithms:

Pathfinding and Puzzle Solving: Stacks are used to remember choices in backtracking algorithms (e.g., maze solving, Sudoku). When a dead-end is reached, the program pops back to a previous state and tries a different path.

Undo Mechanisms in Applications:

- **Text Editors and Software**: Actions performed by the user are pushed onto a stack. When the user requests an undo, the last action is popped off and reversed.

Syntax Parsing:

- **Compilers and Interpreters**: During parsing, stacks help manage nested structures like parentheses, braces, or brackets to ensure proper syntax.

5. **Memory Management**:

- **Dynamic Memory Allocation**: Some programming languages use stacks to manage temporary variables and perform memory allocation during function calls.

Convert Infix to Postfix algorithm

1. **Traversing the Expression**: Scan the given infix expression from left to right and go character by character.

2. **If the Scanned Character is an Operand**: Directly output the operand (e.g., A, B, 3, etc.) to the postfix expression.

3. **If the Scanned Character is an Operator** :

    - If the operator's precedence is greater than the operator on top of the stack, or the stack is empty, or the top of the stack is a (, push the scanned operator onto the stack.

    - **Operator Precedence**: Operators like * and / have higher precedence over + and -. This step ensures that operators with higher precedence are handled first.

4. **If the Scanned Operator has Lower or Equal Precedence**:

    - While the operator at the top of the stack has greater than or equal precedence than the current operator, pop the stack and append each popped operator to the postfix expression.

    - After popping the operators with higher or equal precedence, push the current operator onto the stack.

5. **Parentheses Handling**: If you encounter a (, during popping, stop popping and push the scanned operator onto the stack.

6. **If the Scanned Character is a Left Parenthesis (**:

    - Push the (onto the stack. This helps to keep track of the boundaries of sub-expressions. It ensures that operators are processed correctly within those bounds.

7. **If the Scanned Character is a Right Parenthesis)**:

    - Pop the stack and output each operator until a (is encountered.

    - Eliminate both the (and) after processing.

    - This ensures that any operators inside parentheses are properly evaluated and placed in the correct order.

8. **Repeat Steps 2 to 6**: Continue scanning the entire expression, applying the above steps for each character.

9. **End of Expression**:

    - After the entire expression has been scanned, pop and print any remaining operators from the stack.

    - This step ensures that all operators are output in the correct postfix order.

Program:

```
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#define MAX 20

char stk[20];
int top = -1;

int isEmpty()
{
   return top == -1;
}

int isFull()
{
   return top == MAX - 1;
}

char peek()
{
   return stk[top];
}

char pop()
{
   if (isEmpty())
      return -1;

   char ch = stk[top];
   top--;
   return(ch);
}

void push(char oper)
{
   if (isFull())
      printf("Stack Full!!!!");
   else {
      top++;
      stk[top] = oper;
   }
}
```

```
int checkIfOperand(char ch)
{
   return (ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z');
}

int precedence(char ch)
{
   switch (ch)
   {
   case '+':
   case '-':
      return 1;

   case '*':
   case '/':
      return 2;

   case '^':
      return 3;
   }
   return -1;
}

void convertInfixToPostfix(char* expression)
{
   int i, j = 0;
   char postfix[MAX];

   for (i = 0; expression[i]; ++i)
   {
      if (checkIfOperand(expression[i]))
         postfix[j++] = expression[i];

      else if (expression[i] == '(')
         push(expression[i]);

      else if (expression[i] == ')')
      {
         while (!isEmpty() && peek() != '(')
            postfix[j++] = pop();
         if (!isEmpty() && peek() != '(')
            return; // Invalid expression
         else
            pop();
```

```
        }
        else
        {
            while (!isEmpty() && precedence(expression[i]) <= precedence(peek()))
                postfix[j++] = pop();
            push(expression[i]);
        }
    }

    while (!isEmpty())
        postfix[j++] = pop();

    postfix[j] = '\0';   // Null-terminate the postfix expression

    printf("Postfix Expression: %s\n", postfix);
}

int main()
{
    char expression[] = "((x+(y*z))-w)";
    convertInfixToPostfix(expression);
    return 0;
}
```

Advantage of Postfix over Infix Expression

- Postfix eliminates the need for parentheses. Therefore, it simplifies expression notation.
- Operator precedence is built into the notation. It avoids ambiguity.
- Computers can evaluate postfix expressions using a stack without complex precedence rules.
- Postfix notation is easier for compilers to parse and process.
- The evaluation follows a uniform left-to-right order, which reduces computational complexity.

infix to prefix algorithm,

1. **Reversing the Infix Expression**

The input infix expression is first reversed. During this process, the left parentheses (are swapped with right parentheses) and vice versa.

Example: For ((a/b)+c), the reversed expression becomes)c+(b/a(.

2. **Converting Reversed Infix to Postfix**

The reversed infix expression is then converted to a postfix expression. This conversion follows the standard postfix rules:

Operands (like a, b, c, etc.) are directly added to the output.

Operators (like +, -, *, /) are pushed to a stack based on their precedence.

When encountering a right parenthesis), operators are popped from the stack until a left parenthesis (is found.

The precedence of operators dictates the order of popping. For example, * and / have higher precedence than + and -.

3. **Reversing the Postfix to Get Prefix:**

Finally, the postfix expression obtained from Step 2 is reversed to form the prefix expression.

```
#include<stdio.h>
#include<string.h>
#include<limits.h>
#include<stdlib.h>

#define MAX 100

int top = -1;
char stack[MAX];

// checking if stack is full
int isFull ()
{
  return top == MAX - 1;
}

// checking is stack is empty
int isEmpty ()
{
  return top == -1;
}

void push (char item)
{
  if (isFull ())
    return;
```

```
  top++;
  stack[top] = item;
}

// Function to remove an item from stack.  It decreases top by 1
int pop ()
{
  if (isEmpty ())
    return INT_MIN;

  // decrements top and returns what has been popped
  return stack[top--];
}

// Function to return the top from stack without having to remove it
int peek ()
{
  if (isEmpty ())
 return INT_MIN;
  return stack[top];
}

// A utility function: It is to check if the provided character is operand
int checkIfOperand (char ch)
{
  return (ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z');
}

// Fucntion to compare precedence
// If we return bigger value, it means higher precedence
int precedence (char ch)
{
  switch (ch)
    {
    case '+':
    case '-':
      return 1;

    case '*':
    case '/':
      return 2;

    case '^':
      return 3;
    }
```

```
  return -1;
}

// The driver function for infix to postfix conversion
int getPostfix (char *expression)
{
  int i, j;

  for (i = 0, j = -1; expression[i]; ++i)
    {

      if (checkIfOperand (expression[i]))
            expression[++j] = expression[i];

      else if (expression[i] == '(')
            push (expression[i]);

      else if (expression[i] == ')')
            {
              while (!isEmpty (stack) && peek (stack) != '(')
                expression[++j] = pop (stack);
              if (!isEmpty (stack) && peek (stack) != '(')
                return -1;                  // invalid expression
              else
                pop (stack);
            }
      else                          // if an opertor
            {
              while (!isEmpty (stack)
                        && precedence (expression[i]) <= precedence (peek (stack)))
                expression[++j] = pop (stack);
              push (expression[i]);
            }

    }

  // Once all inital expression characters are traversed
  // adding all left elements from stack to exp
  while (!isEmpty (stack))
    expression[++j] = pop (stack);

  expression[++j] = '\0';

}
```

```
void reverse (char *exp)
{

  int size = strlen (exp);
  int j = size, i = 0;
  char temp[size];

  temp[j--] = '\0';
  while (exp[i] != '\0')
    {
      temp[j] = exp[i];
      j--;
      i++;
    }
  strcpy (exp, temp);
}

void brackets (char *exp)
{
  int i = 0;
  while (exp[i] != '\0')
    {
      if (exp[i] == '(')
            exp[i] = ')';
      else if (exp[i] == ')')
            exp[i] = '(';
      i++;
    }
}

void InfixtoPrefix (char *exp)
{

  int size = strlen (exp);

  // reverse string
  reverse (exp);
  //change brackets
  brackets (exp);
  //get postfix
  getPostfix (exp);
  // reverse string again
  reverse (exp);
}
```

```
int main ()
{
  printf ("The infix is: ");

  char expression[] = "((a/b)+c)-(d+(e*f))";
  printf ("%s\n", expression);
  InfixtoPrefix (expression);

  printf ("The prefix is: ");
  printf ("%s\n", expression);

  return 0;
}
```

Postfix Evaluation Algorithm

Postfix evaluation algorithm is a simple algorithm that allows us to evaluate postfix expressions. The algorithm uses a stack to keep track of operands and performs arithmetic operations when an operator is encountered. The algorithm can be summarized in the following steps:

1. First of all, it will ***Create*** an ***empty stack***.
2. After that, it ***Scan*** the expression from ***left to right***.
3. If an operand is encountered, it ***push*** it onto the stack.
4. If an operator is encountered, ***pop*** the top two operands from the stack, perform the operation, and ***push*** the result back onto the stack.
5. After that, it ***Continue scanning*** the expression until all tokens have been processed.
6. When the expression has been fully scanned, the result will be the top element of the stack.

Program :

```
#include <stdio.h>

#include <stdlib.h>

#define MAX_SIZE 20

// Stack implementation

int stack[MAX_SIZE];

int top = -1;

void push(int item) {

   if (top >= MAX_SIZE - 1) {

printf("Stack Overflow\n");

      return;

   }

   top++;

   stack[top] = item;

}

int pop() {

   if (top < 0) {

printf("Stack Underflow\n");

      return -1;
```

```
    }
    int item = stack[top];
    top--;
    return item;
}
int is_operator(char symbol) {
    if (symbol == '+' || symbol == '-' || symbol == '*' || symbol == '/') {
        return 1;
    }
    return 0;
}
int evaluate(char* expression) {
    int i = 0;
    char symbol = expression[i];
    int operand1, operand2, result;

    while (symbol != '\0') {
        if (symbol >= '0' && symbol <= '9') {
            int num = symbol - '0';
            push(num);
        }
        else if (is_operator(symbol)) {
            operand2 = pop();
            operand1 = pop();
            switch(symbol) {
                case '+': result = operand1 + operand2; break;
                case '-': result = operand1 - operand2; break;
                case '*': result = operand1 * operand2; break;
                case '/': result = operand1 / operand2; break;
            }
```

```
            push(result);
        }
    i++;
        symbol = expression[i];
    }
    result = pop();
    return result;
}
int main() {
    char expression[] = "5 6 7 + * 8 -";
    int result = evaluate(expression);
printf("Result= %d\n", result);
return 0;
}
```

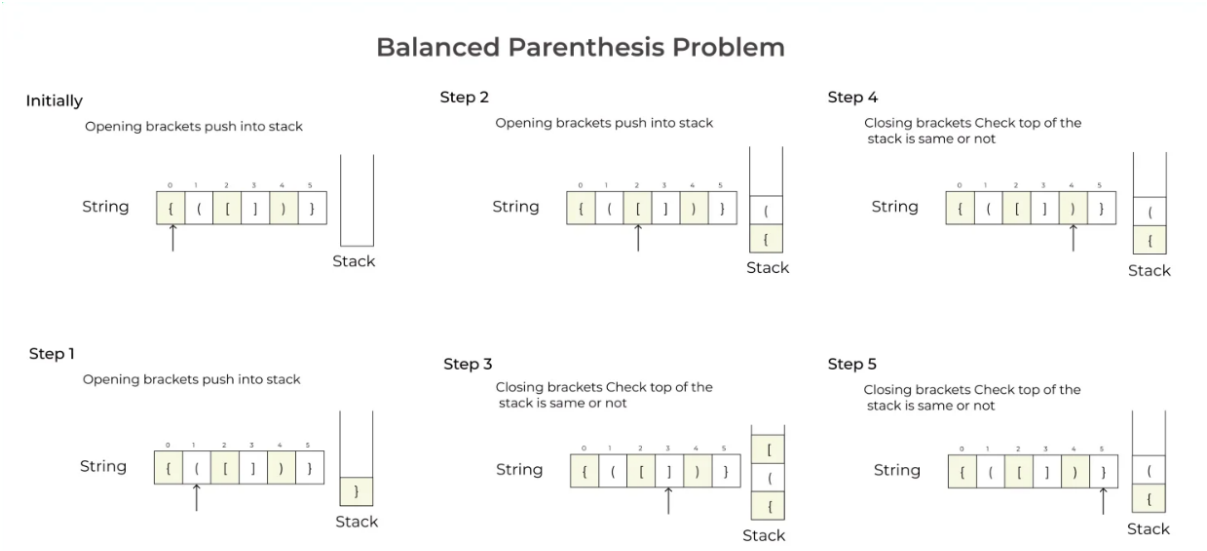
Output:

57

Stack Applications[Validating the Parathesis]

Algorithm :

- Declare a structure for character stack.
- Now traverse the expression string exp.
    1. If the current character is a starting bracket (**'(' or '{' or '['**) then push it to stack.
    2. If the current character is a closing bracket (**')' or '}' or ']'**) then pop from stack and if the popped character is the matching starting bracket then fine else brackets are not balanced.
- After complete traversal, if there is some starting bracket left in stack then "NOT BALANCED"



Program

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
struct stack {
  char stck[MAX];
  int top;
}s;

void push(char item) {
  if (s.top == (MAX - 1))
    printf("Stack is Full\n");
  else {
    s.top = s.top + 1;
    s.stck[s.top] = item;
  }
}
```

```
void pop() {
  if (s.top == -1)
    printf("Stack is Empty\n");

  else
    s.top = s.top - 1;
}

int checkPair(char val1,char val2){
    return (( val1=='(' && val2==')' )||( val1=='[' && val2==']' )||( val1=='{' && val2=='}' ));
}

int checkBalanced(char expr[], int len){

    for (int i = 0; i < len; i++)
    {
        if (expr[i] == '(' || expr[i] == '[' || expr[i] == '{')
        {
          push(expr[i]);
        }
        else
        {
            if (s.top == -1)
            return 0;
        else if(checkPair(s.stck[s.top],expr[i]))
        {
            pop();
            continue;
        }
        return 0;
        }
    }
    return 1;
}
int main() {
  char exp[MAX] = "({})[]{}";
  int i = 0;
  s.top = -1;
  int len = strlen(exp);
  checkBalanced(exp, len)?printf("Balanced"): printf("Not Balanced");

  return 0;
}
```

Output:

Balanced

Reversing List

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
int stack[MAX];
int top = -1;
void push(int value);
int pop();
void display();
void reverseStack();
void push(int value)
 {
  if(top >= MAX - 1)
 {
        printf("Stack Overflow! Cannot push %d\n", value);
        return;
    }
    stack[++top] = value;
    printf("%d pushed onto the stack.\n", value);
}
// Function to pop an element from the stack
int pop()
    {
    if(top < 0)
      {
        printf("Stack Underflow! Stack is empty.\n");
        return -1;
    }
    return stack[top--];
}

// Function to display the stack elements
void display() {
    if(top < 0) {
        printf("Stack is empty.\n");
        return;
    }
    printf("Stack elements: ");
    for(int i = top; i >= 0; i--) {
        printf("%d ", stack[i]);
    }
    printf("\n");
}

// Function to reverse the stack
void reverseStack() {
    if(top < 0) {
        printf("Stack is empty. Nothing to reverse.\n");
        return;
```

```
    }

    int start = 0, end = top;
    while(start < end) {
        // Swap the elements at start and end
        int temp = stack[start];
        stack[start] = stack[end];
        stack[end] = temp;
        start++;
        end--;
    }
    printf("Stack has been reversed.\n");
}

int main()
 {
    int choice, value;
       while(1) {
        printf("\nStack Operations using Arrays\n");
        printf("1. Push\n2. Pop\n3. Display\n4. Reverse Stack\n5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;
            case 2:
                value = pop();
                if(value != -1)
                    printf("Popped element: %d\n", value);
                break;
            case 3:
                display();
                break;
            case 4:
                reverseStack();
                break;
            case 5:
                printf("Exiting program...\n");
                exit(0);
            default:
                printf("Invalid choice! Please try again.\n");
        }
    }
   return 0;
}
```

Output:

Stack Operations using Arrays
1. Push
2. Pop
3. Display
4. Reverse Stack
5. Exit
Enter your choice: 1
Enter value to push: 10
10 pushed onto the stack.

Stack Operations using Arrays
1. Push
2. Pop
3. Display
4. Reverse Stack
5. Exit
Enter your choice: 1
Enter value to push: 20
20 pushed onto the stack.

Stack Operations using Arrays
1. Push
2. Pop
3. Display
4. Reverse Stack
5. Exit
Enter your choice: 1
Enter value to push: 30
30 pushed onto the stack.

Stack Operations using Arrays
1. Push
2. Pop
3. Display
4. Reverse Stack
5. Exit
Enter your choice: 3
Stack elements: 30 20 10
Stack Operations using Arrays
1. Push
2. Pop
3. Display
4. Reverse Stack
5. Exit
Enter your choice: 4
Stack has been reversed.

Stack Operations using Arrays
1. Push
2. Pop
3. Display
4. Reverse Stack
5. Exit
Enter your choice: 3
Stack elements: 10 20 30

Stack Operations using Arrays
1. Push
2. Pop
3. Display
4. Reverse Stack
5. Exit
Enter your choice: 5
Exiting program...

Back Tracking [Example]

Depth First Search (DFS)

DFS uses Stack.

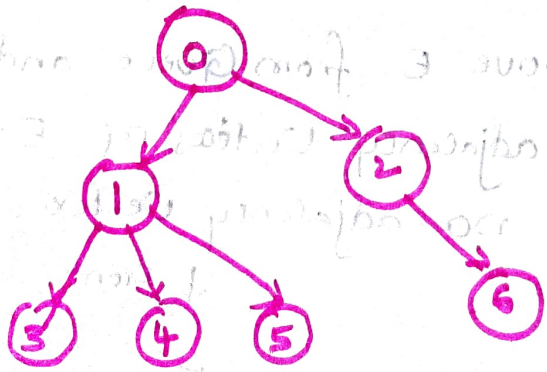
Initially Stack is empty and Top = -1

Initially VISITED[i] = False where i = 1 to n,

n is total no. of nodes.

procedure:

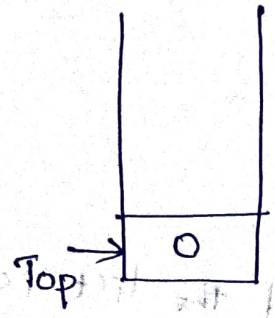
1. push starting node into stack.
2. pop an element from the stack, if it has not been traversed then traverse it, After traveresing make the value of visited array true for this node.
3. push all the unvisited adjacent nodes of the poped element on stack.
4. Repeat Step 3 and 4 until stack is empty.



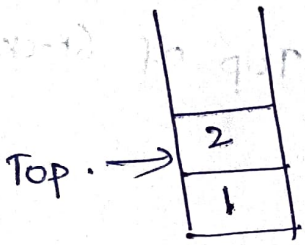
DFS.

Step1:

push 0 into Stack, initially.

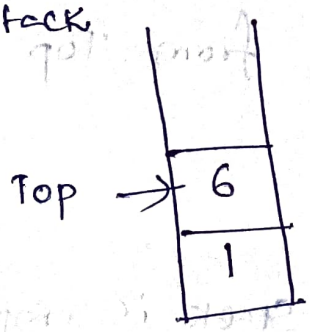


Step2: push into the stack the neighbours of the vertex currently being processed and pop the top of the stack.



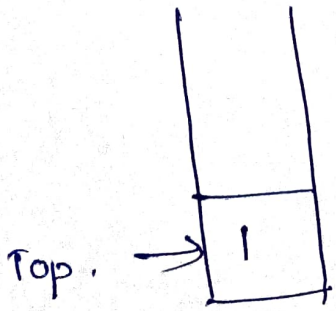
Neighbours of 0 is 1 and 2

Step3: pop 2 and push neighbours of 2 into Stack



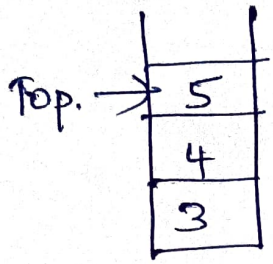
Neighbours of 2 is 6.

Step4: pop 6 and push neighbours of 6. Here there is no neighbour to 6. So Stack will be same. and pop 6.

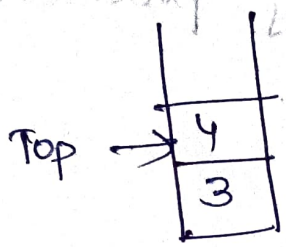


Step5:

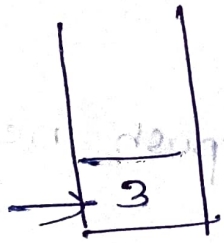
pop 1 and push neighbours of 1 are 3,4,5 into Stack.



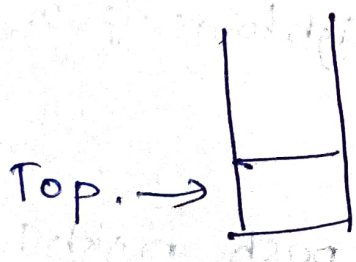
Step6. pop 5 from Top. and the list is



Step 7: pop 4 from Top of Stack



Step 8: pop 3 from Top of Stack.



Stack is empty

Stop the procedure.

DFS order is: 0, 2, 6, 1, 5, 4, 3.

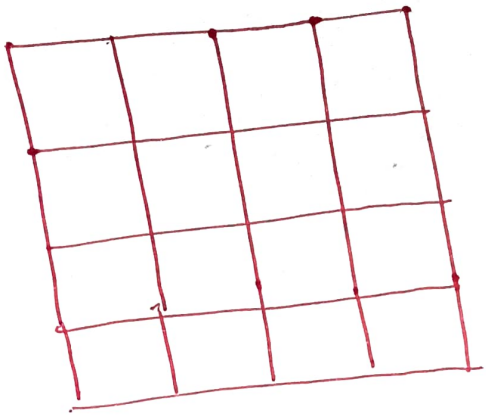
Example : 2

N Queen problem.

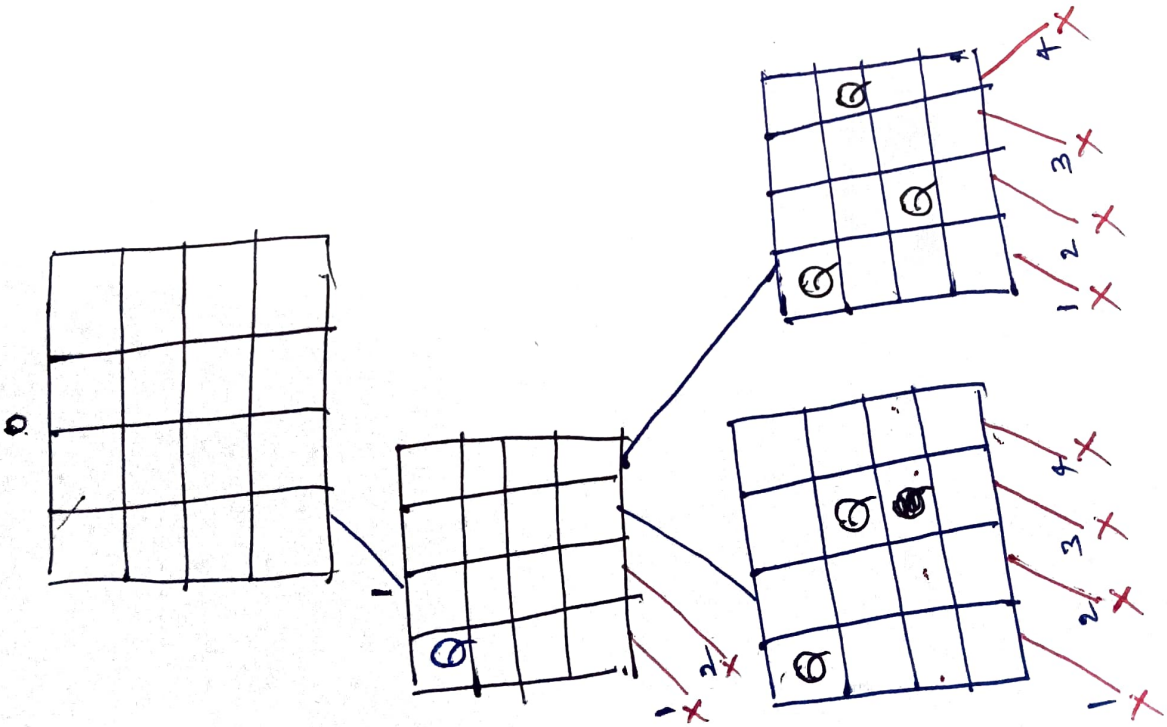
The problem is to place n queens on an nxn chess board. So that no two Queens attack each other. by being in the Same row. or in the Same Column or on the Same diagonal.

4 Queen problem.

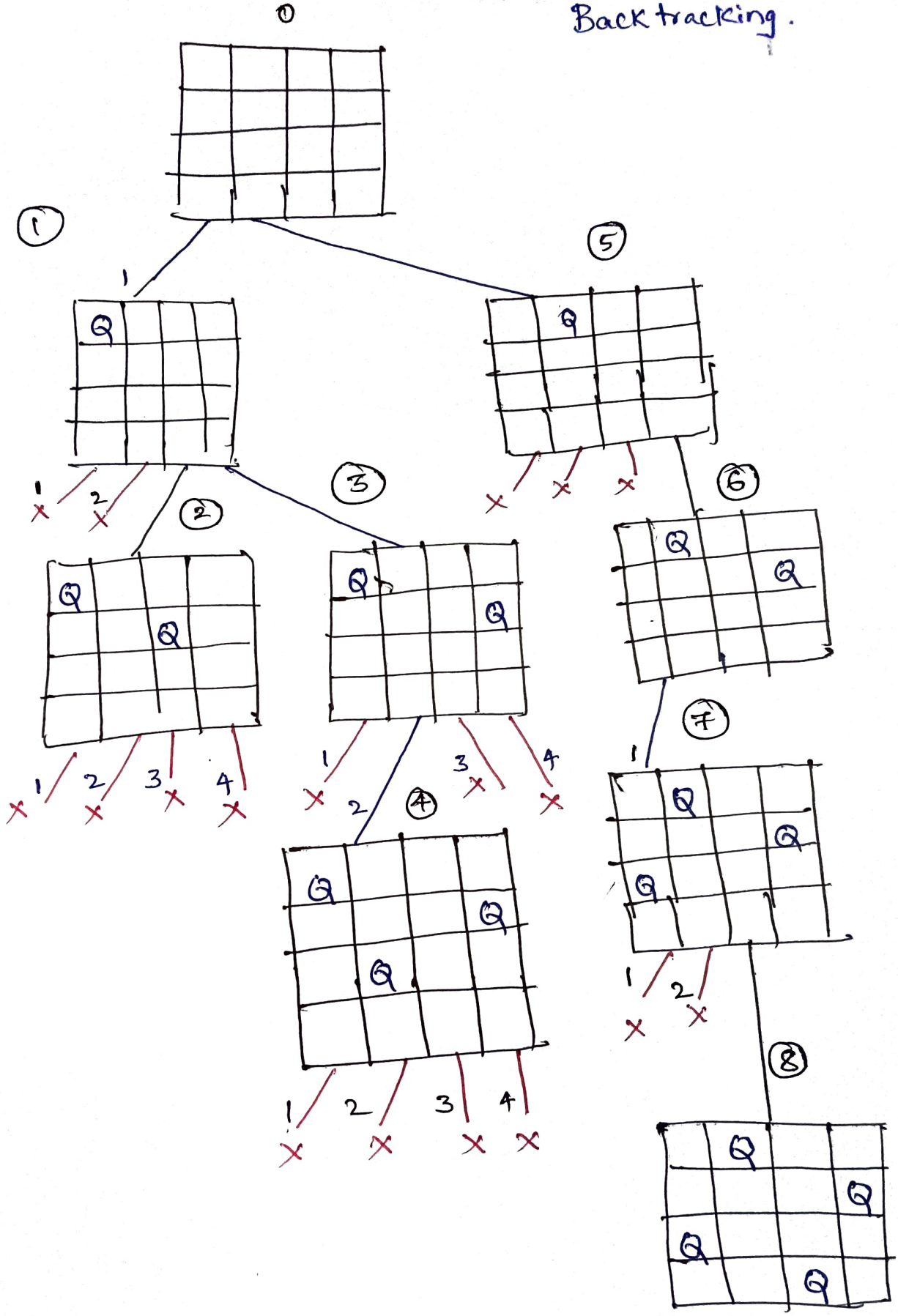
4x4 chess board.



Solution.



Backtracking.



Solution.